

VIPEX Coach — AI Engineer Case Study

Role: AI Engineer – VIPEX Coach (GCC) **Duration:** ~2.25 hours **Format:** Take-home

Instructions for Candidate

You are being considered for the AI Engineer role on **VIPEX Coach**, an internal Carlsberg AI product. After the Asia pilot, the GCC team will own the system end-to-end: agent runtime, knowledge base, evaluation, Teams integration, and post-handover operations.

This case study simulates the kind of work you'd face in the first 90 days. Read the problem brief carefully, then complete **Parts A–D**. You may use any tools (Python, diagrams, pseudocode) to support your answers. We value clarity of thinking over polish.

Note actual time per section so we know where you spent your effort. We are explicitly more interested in your reasoning and trade-off thinking than in exhaustive coverage — if a question feels under-specified, state your assumption in one line and proceed. Open-ended questions are open on purpose. There is no single right answer to most of what follows; there are answers that show how you'd attack a real piece of work, and answers that don't. We grade for the former.

Problem Brief: VIPEX Coach

What is VIPEX Coach?

VIPEX Coach is an AI engine that accelerates Carlsberg's packaging cost-saving process. Markets run periodic value-engineering workshops — *Walk the Floor, Gemba Kaizen, Portfolio Simplification* — where commercial, supply-chain, and finance teams brainstorm packaging ideas: gauge reductions, neck-finish standardisation, label simplifications, secondary-packaging swaps, SKU consolidations.

The raw output of those workshops is messy. Ideas arrive as handwritten sticky-notes, QR-code submissions on mobile, Excel exports in local language, screenshots of whiteboards. A team of three to five people then spends two to four weeks per market translating them into English, deduplicating across workshops, scoping savings, and turning each idea into a one-pager business can decide on.

VIPEX Coach replaces the manual middle. It ingests raw inputs, cleans and translates them, deduplicates across submissions and markets, estimates savings as sensitivity bands, scores feasibility against historical rejection patterns, and generates implementation-ready one-pagers. Markets interact with it through a Microsoft Teams bot.

Business Context

Metric	Today	Target
Pipeline leakage (ideas that die before becoming proposals)	~30%	<10%
Manual curation per market per workshop	2–4 weeks	<1 day end-to-end
Unrealised savings, Carlsberg-wide, annualised	~80M DKK	–
2026 validated savings target, Asia MVP	–	~10M DKK

The Manual Process Today

A market business analyst receives a workshop output: raw ideas, mostly in Vietnamese or Chinese, often duplicated, often vague ("*look at can gauge*"). They open Excel, hand-translate, hand-merge similar ideas, look up affected SKUs and material spend in ERP exports, sketch a

savings range. They walk the result through brand teams and procurement. Two to four weeks later, a shortlist of ideas with cost estimates is ready for commercial steerco review.

Many ideas are rejected at this stage — sometimes for technical reasons, often for *tacit* commercial reasons that aren't written down anywhere: *"China carton doubles as in-store promotional display — you can't substitute a tray"*, *"Malaysia is a Muslim market, no coloured glass"*, *"China labels wort concentration, not ABV — your savings comparison is wrong"*. The rejection knowledge lives in people's heads.

The Asia MVP

The Asia MVP launches in Vietnam (pilot) and China (priority) in mid-2026. After launch, the GCC team takes ownership from the build team: knowledge-transfer in June, full handover July–August. The GCC team owns the runtime, the knowledge base, evaluation, and incident response.

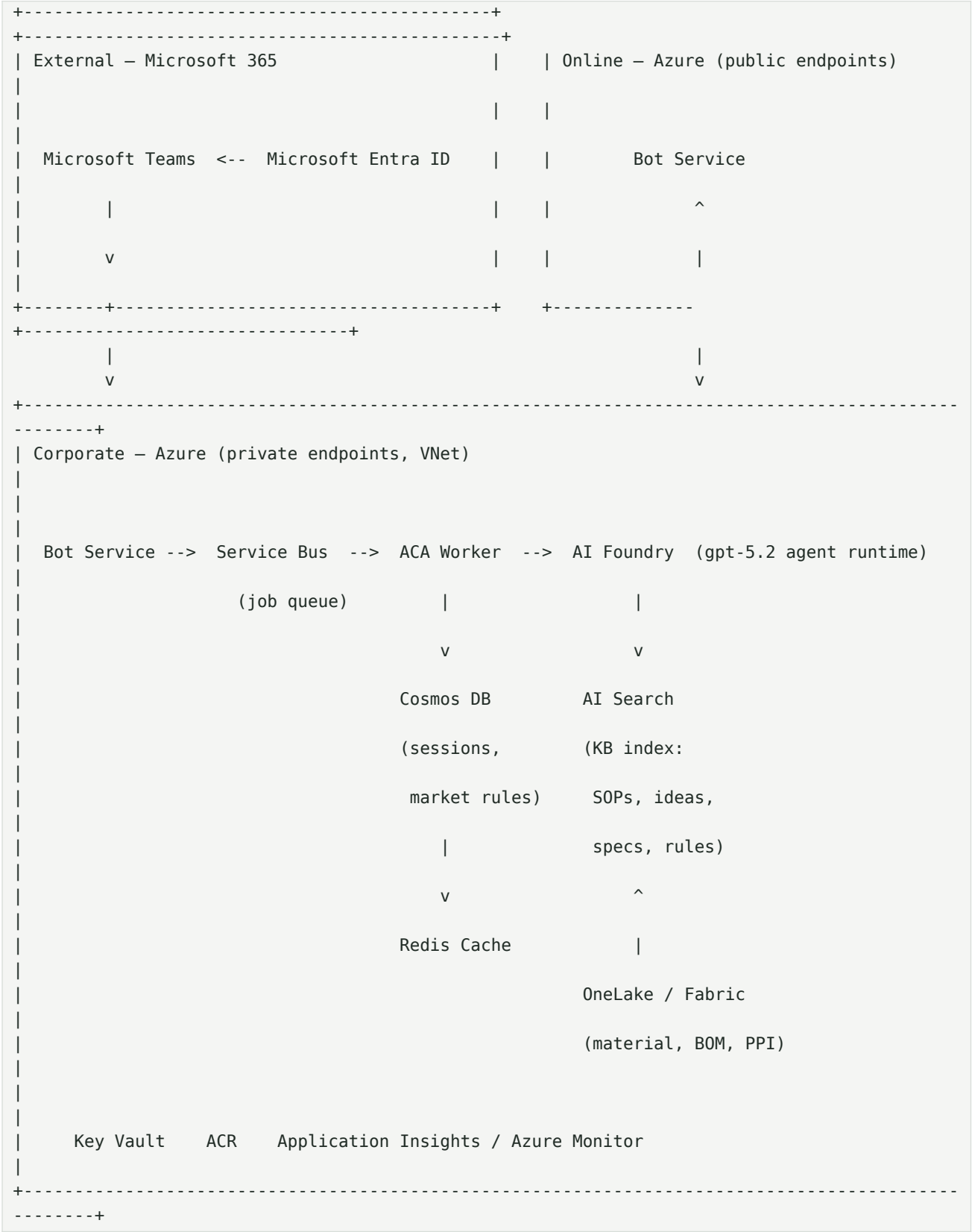
Why This Matters for the Hire

The successful candidate will inherit a live system within weeks of starting. You will not design VIPEX Coach from scratch — you will own it. The case study below tests how you think about (a) the agent architecture you're inheriting, (b) the Teams integration patterns the bot already uses, (c) the evaluation framework that keeps the agent honest after handover, and (d) the operational reality of being on-call for a multi-market AI product.

What Success Looks Like in Year 1

Twelve months after handover, the team that wins this role is running a stable agent across two markets with measurable, defensible savings flowing into the commercial steering committee. Specifically: pilot acceptance rates above 70%, end-to-end workshop-file processing under two minutes at the 95th percentile, an eval suite that catches regressions before they reach production, and a market-rule memory base that grows with the markets it serves rather than rotting silently. The hire is expected to drive each of those — instrument, evaluate, debug, ship. The role is hands-on. Architecture documents are useful only if they map to code, telemetry, and Teams interactions the user can see.

Technical Architecture



Component Inventory

Service	Purpose	Zone	Notable Config
Microsoft Teams	User-facing chat surface for market BAs and analysts	External (M365)	File upload, adaptive cards
Bot Service	Receives Teams activities, stores conversation references, queues jobs, and sends results back into the same Teams thread	Online (public endpoint)	Managed identity to Service Bus
Service Bus	Async job queue between Bot and Worker; dead-letter for poison messages	Corporate (private)	Premium SKU
ACA Worker	Container Apps Job: picks jobs off Service Bus, invokes agent, writes results	Corporate (private)	Auto-scale 1–10 replicas
AI Foundry	Agent runtime — orchestrates cleaning, idea generation, one-pager drafting, and market-memory lookups against tools and KB	Corporate (private)	gpt-5.2, content safety, approved outbound only
AI Search	Hybrid (vector + keyword) index over SOPs, prior ideas, material specs, and market rules	Corporate (private)	Semantic ranker enabled
Cosmos DB	Conversation/session state; idea history; market-rule store	Corporate (private)	SQL API
Redis Cache	Idempotency keys; rate limits; hot session reads	Corporate (private)	Premium, zone-redundant
OneLake / Fabric	Source of truth for material masters, BOMs, supplier PPI, cross-market benchmarks	Corporate (private)	Read-only from agent
Key Vault, ACR, App Insights	Secrets, container registry, observability	Corporate (private)	Managed identity throughout

Data Flow

1. User sends a message in Microsoft Teams (chat message or file upload).
2. Bot Service receives the activity, validates the payload, stores the *conversation reference*, queues a job to Service Bus with the message body, and returns an immediate acknowledgement to Teams. The bot does **not** wait for the agent.
3. Worker picks up the job from the queue (one job per replica, message lock held).
4. Worker invokes the AI Foundry agent with a structured request: user intent, market, brand, attached files, session reference.
5. Agent runs against the KB (AI Search) and structured data (OneLake), persists intermediate state (Cosmos), and produces a response.
6. Worker writes results back to Cosmos and uses the stored conversation reference to send the final result back into the same Teams thread once processing completes.
7. User sees the result inline in Teams. Replies continue the cycle — session continuity is read from Cosmos on the next turn.

Agent Runtime

The Asia MVP runs on AI Foundry with a gpt-5.2 deployment. In practice it uses a pragmatic agentic pipeline with persisted state, structured tool / service calls, and asynchronous job execution. The exact orchestration is intentionally abstracted for this exercise: focus on capability boundaries and operational constraints rather than reverse-engineering a specific scaffold. Representative capabilities exposed to the runtime include:

Tool	What it does	Typical caller
<code>retrieve_material(sku, market)</code>	Returns material master, BOM, supplier PPI for a given SKU and market from OneLake.	Idea cleaning / savings estimation, idea generation
<code>find_similar_ideas(query, market, k)</code>	Hybrid vector + keyword search across prior ideas and SOPs in AI Search.	Deduplication, cross-market benchmarks
<code>score_feasibility(idea, market)</code>	Returns a 0–100 score with reason codes, using market rules + rejection history.	Feasibility scoring
<code>compute_savings_band(idea, market)</code>	Returns low / mid / high savings tied to material spend, not a single number.	Savings estimation
<code>generate_alternatives(idea, n)</code>	Returns n fallback variants of a primary proposal.	Idea generation
<code>write_rule(rule_text, market, category)</code>	Captures a tacit rule from conversation; persists in Cosmos with provenance + validity.	Market memory capture
<code>render_one_pager(idea, audience)</code>	Produces an implementation-ready brief — audience controls the depth and tone.	One-pager drafting

One Foundry-facing runtime coordinates these capabilities for the Asia MVP, with session and idea history persisted to Cosmos and market isolation enforced primarily at the data layer (per-market retrieval filters, scoped memory, and market-tagged rules). The important design question for this case is not the exact internal decomposition, but where you would draw boundaries between deterministic logic, tools / services, and model judgment.

Part A: Problem Diagnosis & Ops

A.1 Root Cause Structuring

Categorise the most likely failure modes of VIPEX Coach in pilot into **Data**, **Method**, and **Operating Model** root causes. One row per category — fill the table. The italicised anchors are starting points; challenge or replace them as you see fit.

Data anchor: material-master coverage is uneven across markets. Vietnam has ~6,000 SKUs mapped; China has ~12,000 but many lack supplier PPI. Method anchor: savings are returned as bands, but commercial steerco still pushes back ("which number do I commit to?"). Operating-Model anchor: market rules are tagged per market, but the SME who captured them has no clear review cadence — rules age silently.

Category	Root Cause	Symptom(s) it explains
Data	Incomplete or inconsistent master data (missing SKU mappings, supplier PPI, outdated BOMs) across markets	Incorrect savings estimates, inability to retrieve material information, inconsistent outputs between Vietnam and China
Method	Savings estimation and feasibility algorithms are not sufficiently calibrated to business decision-making; model outputs lack confidence and explainability	Commercial teams reject AI recommendations despite accurate data; users distrust savings bands; inconsistent feasibility scores
Operating Model	No governance for maintaining market rules, validating AI outputs, or incorporating SME feedback	Outdated market memory, repeated recommendation of rejected ideas, declining user trust and pilot adoption

A.2 Hypothesis Validation

Suppose "*ideas are accepted by the agent but rejected by commercial steerco*" is the dominant complaint after the Vietnam pilot. You suspect the issue is in **savings-band methodology**, not in the underlying material-spend data. What data or experiment would you request to confirm or refute that hypothesis? Be specific about the metric you'd compare, the time window, and what "confirmed" would look like.

ANSWER:

Hypothesis:

Issue is in savings-band methodology.

Data I would request:

1. Material master values: To verify that inputs were actually correct
2. AI outputs: prediction, confidence, explanation
3. Commercial review: accepted/rejected, reason

Metrics:

1. Acceptance rate by savings confidence
2. Absolute error between Mid estimate and Finance estimate
3. Band width High-Low vs Probability of rejection

Time window:

Analyze every idea processed during the Vietnam pilot (for example, the first 6–8 weeks after launch).

Confirmation:

- If more than 80% of rejected ideas have accurate underlying material data, but the predicted savings bands are consistently much wider than finance estimates or fail to align with the values ultimately used by the commercial team, then the hypothesis is supported.
- If the material data itself is frequently incomplete or incorrect, the hypothesis is refuted because the root cause lies in data quality rather than methodology.

A.3 Prioritisation

You can reduce only **one** launch risk before the Vietnam pilot. Choose from: poor data coverage, weak savings credibility, bad feasibility judgments, weak duplicate handling, low-quality market-memory capture, multilingual normalization / translation errors, weak user feedback capture, or unreliable end-to-end processing. Which risk would you reduce first, and why? What risk are you explicitly accepting by not choosing the others?

ANSWER:

- | | | |
|---|------------------------|----------------------------------|
| 1 | System reliability | Unreliable end-to-end processing |
| 2 | Data quality | Poor data coverage |
| | | Translation |
| | | Duplicate handling |
| 3 | Decision quality | Savings credibility |
| | | Feasibility |
| | | Market memory |
| 4 | Continuous improvement | User feedback |

I would first reduce: "Unreliable end-to-end processing"

Because: A correct AI that is not reliable is useless. Working with unreliable piece of software is like playing bet.

By making this choice, I am explicitly accepting other launch risks. I accept that duplicate detection and multilingual normalization may not be perfect, resulting in some additional manual effort. I also accept that market-memory capture and user feedback mechanisms will initially be limited, slowing long-term learning. These issues affect efficiency and continuous improvement, but they are less alarming / critical than "Unreliable end-to-end processing" to undermine stakeholder trust in the pilot.

A.4 Operational Incident

Scenario. It is 02:00 local time in Vietnam. A market SME uploaded a 300-idea workshop file into the Teams bot ninety minutes ago. The bot acknowledged the upload and posted a "*processing your file...*" message, but no result has come back. The user is now on Teams asking for help. You are on-call from the GCC team.

Walk through your first hour of triage: what do you check, in what order, and why. Reference the architecture above. **Useful surfaces to consider:** Service Bus DLQ depth and active message count, ACA Worker logs and replica status, AI Foundry run status (running / failed / requires-action), Cosmos session record for that conversation, the stored Teams conversation reference, Foundry rate limits and token consumption on the gpt-5.2 deployment.

We are testing reasoning, not Azure trivia recall — a defensible answer beats an exhaustive one.

ANSWER:

Step 1 -- Acknowledge the user

Before investigating, I would update the user in Teams:

"Thanks for reporting this. I can confirm the upload was received, but processing appears to be delayed. I'm investigating the pipeline now and will provide an update within 15–20 minutes."

Step 2 -- Check Service Bus

This is the first checkpoint.

Questions:

- Is the message still waiting?
- Did it move to the Dead Letter Queue (DLQ)?
- Is there a backlog?

Possible observations:

Case A: Active Queue

Worker isn't consuming.

Likely ACA Worker issue.

Case B: DLQ

Processing failed before reaching AI.

Case C: Queue empty

The worker picked it up.

Issue is somewhere downstream.

Step 3 -- ACA Worker Health

Now verifying the workers.

Questions:

- Are replicas running?
- Any crash loops?
- Any out-of-memory errors?
- Did autoscaling fail?
- Are jobs stuck?

Step 4 -- AI Foundry Run Status

If the worker is healthy, check the AI run.

Questions:

Running? Failed? Requires action? Timed out?

Possible cases: Running, Maybe processing a very large file, Need progress, Failed

Read error: It maybe because of tool failure, prompt failure, or timeout

Requires Action: Perhaps a tool call failed; Worker waiting forever

This is a classic agent failure mode.

Step 5 -- Check Rate Limits and Token Consumption

The input contains 300 ideas. That is enormous.

Questions:

Did GPT-5.2 hit TPM limits?

Too many concurrent runs?

Token quota exhausted?

Step 6 -- Cosmos Session State

Now inspect the conversation record.

Question: Did the worker write progress?

Example:

Status: Processing or Completed

If Cosmos says: Completed

But user received nothing, processing isn't the issue. Delivery is.

Cosmos becomes the source of truth for pipeline state.

Step 7 -- Teams Conversation Reference

The architecture stores conversation references.

Check:

Is the Teams conversation ID valid?

Was it deleted?

Has the thread expired?

Did posting back fail?

Step 8 -- Decide Immediate Recovery

Report the user on where the failure occurred.

Step 9 -- Inform the user

Explain: issue identified, expected resolution, ETA

Part B: Agent Design & Teams Integration

B.1 Capability Boundaries

VIPEX Coach must deliver four classes of capability:

- **Idea cleaning and feasibility preparation** — clean / translate / deduplicate raw workshop ideas; estimate savings as sensitivity bands; score feasibility (0–100).
- **Idea generation** — generate net-new cost-saving ideas from data patterns and cross-market benchmarks; rejection-aware; produce two to three alternatives per idea.
- **One-pager drafting** — generate implementation-ready one-pagers for brand owners and commercial steerco.
- **Market Knowledge Memory** — capture tacit commercial rules via conversation, persist with structured fields (market, category, constraint, rationale, validity), and apply them to feasibility scoring and rejection-aware generation.

You inherit a system that mixes deterministic services, retrieval, persisted memory, and model-driven reasoning. For each job below, decide whether it should live primarily in:

- deterministic code / service logic
- model judgment

Use the following jobs-to-be-done as your answer surface:

8. **Cleaning and deduplicating messy workshop ideas** — mixed language, inconsistent formatting, near-duplicates.
9. **Estimating savings bands** — tied to source data, but often with incomplete information.
10. **Scoring feasibility** — informed by historical rejection patterns, but without hard-coding every business rule.
11. **Drafting one-pagers** — useful enough for a human stakeholder to react to, edit, and circulate.
12. **Capturing and applying market rules** — new tacit rules arrive in conversation and must be stored and reused safely.

Choose the **three hardest jobs** and answer each one explicitly:

- which bucket you would place it in (deterministic logic or model judgment),
- why,
- what the main failure mode would be if you placed it incorrectly,
- and how you would actually handle or mitigate that challenge in the system.

We are less interested in what architecture label you use than in the reasoning you use to place each job on the right side of the boundary.

Useful criteria: repeatability, auditability, latency, cost, eval tractability, multilingual ambiguity, and business risk if wrong.

State explicitly:

- (a) which boundaries you would harden first in the next iteration,
- (b) which areas you would keep flexible,
- (c) which decisions you would defer until production evidence forces a change.

ANSWER:

My guiding principle is:

Use deterministic services wherever correctness, repeatability, auditability, or compliance are critical. Use model judgment where ambiguity, language understanding, or creativity is required.

This keeps the system reliable while leveraging the strengths of LLMs.

Job: Cleaning & Deduplication

Primary Owner: Model Judgment + deterministic validation

Reason: Language ambiguity and semantic similarity require LLM reasoning; deterministic normalization and similarity thresholds improve consistency.

Job: Savings Band Estimation

Reason: Deterministic Service Logic

Reason: Financial calculations must be reproducible, explainable, and auditable.

Job: Feasibility Scoring

Primary Owner: Hybrid (primarily Model Judgment)

Reason: Requires reasoning over historical patterns and tacit business knowledge while using deterministic retrieval and rule enforcement.

Job: One-Pager Drafting

Primary Owner: Model Judgment

Reason: Primarily a natural-language generation task.

Job: Market Rule Capture

Primary Owner: Hybrid (Model + deterministic persistence)

Reason: LLM extracts rules from conversations; deterministic code validates, versions, and stores them.

Three Hardest Jobs

1. Feasibility Scoring

Primary Bucket

Model Judgment (supported by deterministic retrieval)

Why?

Feasibility depends on many factors:

- historical rejection patterns
- market-specific commercial practices
- packaging constraints
- tacit SME knowledge

These cannot realistically be expressed as fixed rules.

The LLM is better suited to combine these signals into a recommendation.

Deterministic components should only retrieve:

- market rules
- rejection history
- material constraints
- structured product information

The model synthesizes these into a score with reasoning.

Failure if implemented deterministically

- If every rule is hard-coded, maintenance becomes expensive, new market rules require deployments, the system cannot adapt to new markets.
- Eventually thousands of special cases accumulate.

Failure if implemented entirely by the model

- The score becomes inconsistent.

- The same idea may receive different feasibility scores across runs.
- Business loses trust.

Mitigation

I would use a hybrid pipeline:

Retrieve market rules >> Retrieve rejection history >> Retrieve material constraints >>

LLM produces:

- feasibility score
- confidence
- explanation >>

Validate score range >> Persist reasoning

This provides explainability while keeping the reasoning flexible.

2. Capturing and Applying Market Rules

Primary Bucket

Hybrid (Model Judgment + Deterministic Persistence)

Why?

Humans express rules naturally.

Example:

"Brand C cartons are also promotional displays."

The LLM should identify:

- Market
- Category
- Constraint
- Business rationale
- Validity period

However, the LLM should never write directly into the production knowledge base.

Failure if entirely deterministic:

- Someone must manually enter every rule.

- Knowledge capture becomes slow.
- Tacit knowledge is lost.

Failure if entirely model-driven:

- The LLM may invent rules,
- modify existing rules,
- or overwrite important knowledge.

This creates governance problems.

Mitigation

I would introduce a review workflow:

Conversation >> LLM extracts structured rule >> Human approval >> Versioned storage >> Activation

Each rule stores:

- source conversation
- creator
- confidence
- review status
- valid_from
- valid_to
- version

This makes the knowledge base auditable.

3. Cleaning and Deduplicating Ideas

Primary Bucket

Model Judgment (supported by deterministic preprocessing)

Why?

The workshop data is messy:

- Vietnamese
- Chinese
- English
- spelling mistakes

- abbreviations
- handwritten OCR
- near-duplicates

Example:

- Reduce can gauge
- 330ml gauge reduction
- Reduce thickness of 330ml aluminum can

These are semantically identical.

Vector similarity alone is insufficient.

An LLM understands intent.

Failure if deterministic only:

- Rule-based matching misses many duplicates.
- Simple cosine similarity also misses multilingual paraphrases.
- Duplicate ideas continue through the pipeline.

Failure if model only:

The LLM may merge genuinely different ideas.

These appear related but have different engineering implications.

Mitigation

Pipeline:

Normalize language >> Translate >> Embedding search >> Top-K candidates >> LLM duplicate decision >> Human review if confidence low

This reduces cost while maintaining precision.

Boundaries I'd Harden First

I would convert these into deterministic, well-tested services first:

1. Savings Band Estimation

Financial outputs must be:

- reproducible

- explainable
- auditable

The LLM should not calculate money.
It should explain the calculation.

2. Rule Persistence

Knowledge management requires:

- version control
- approvals
- expiry
- provenance
- audit trails

This belongs in software, not prompts.

3. Retrieval

Retrieval should remain deterministic:

Market filter >> SKU filter >> Vector search >> Ranking >> LLM

The model should never decide what database records to retrieve.

Areas I'd Keep Flexible

These benefit from continuous LLM improvement:

- idea generation
- one-pager drafting
- multilingual normalization
- duplicate reasoning
- explanation generation

These naturally improve as models improve.

Decisions I'd Defer

Some architectural decisions should be based on production evidence rather than assumptions:

1. Duplicate detection

If embedding search already achieves high precision, there may be no need for an expensive LLM validation step.

2. Feasibility scoring

Initially I would use retrieval plus prompting.

If production data grows, I might replace or augment this with a learned ranking model calibrated on historical accept/reject outcomes.

3. Language routing

I would start with translate >> retrieve >> generate.

If retrieval quality suffers, I would evaluate multilingual embeddings or a mixed-language index before redesigning the pipeline.

My boundary decisions are driven by six engineering principles:

Deterministic

Repeatability

Auditability

Business Risk

Model

Language Understanding

Creativity

Handling Ambiguity

B.2 Knowledge and Retrieval

How would you organise the KB across material masters, BOMs, supplier PPI, prior workshop ideas, and market-memory rules? Walk through retrieval for a single raw idea — "*reduce can gauge on 330ml standard SKUs*":

13. What structured context does the agent extract first? (SKU range, market, brand, packaging component, ...)

14. How does the agent query the KB — pure vector, hybrid, structured pre-filter then vector?

15. How are results ranked and filtered across market rules (market-scoped), prior ideas (cross-market), and material data (market-scoped)?

16. What is the fallback when retrieval returns zero useful hits?

The pass criterion is **correct market scoping**: a Vietnam idea should use Vietnam-specific material data and market rules unless cross-market transfer is explicitly justified.

ANSWER

Overall KB Organization

I would organize the knowledge base into multiple logical stores rather than a single vector index.

Knowledge Source	Storage	Retrieval Type	Scope
Material Masters	OneLake / SQL	Structured lookup	Market-specific
BOM & Supplier PPI	OneLake	Structured lookup	Market-specific
Prior Workshop Ideas	AI Search (Hybrid Index)	Hybrid (keyword + vector)	Cross-market
Market Rules	Cosmos DB + AI Search	Structured filter + semantic	Market-specific
Session Memory	Cosmos DB	Key lookup	Conversation-specific

Material data is structured and should not be embedded.
Prior ideas benefit from semantic search.
Market rules require both metadata filtering and semantic matching.

Retrieval Example

Suppose the user submits:
"Reduce can gauge on 330ml standard SKUs."

The retrieval pipeline would look like this:

- >> User Idea
- >> Context Extraction
- >> Structured Retrieval
- >> Semantic Retrieval
- >> Ranking
- >> LLM Reasoning

Structured Context Extraction

The first step is not vector search.

The agent first extracts structured entities from the idea.

For this example:

Entity :: Extracted Value

Market :: Vietnam

Packaging Component :: Aluminum Can

Volume :: 330ml

Idea Type :: Gauge Reduction

Candidate SKU Range :: All VN 330ml can SKUs

Brand If specified, otherwise infer later

The extracted entities become retrieval filters.

Example JSON:

```
{  
  "market": "VN",  
  "package_type": "can",  
  "size": "330ml",  
  "idea_category": "gauge_reduction",  
  "candidate_skus": ["SKU_VN_3301", "SKU_VN_3302"]  
}
```

This reduces unnecessary retrieval and improves accuracy.

Query Strategy

I would not use pure vector search.

Instead I would use:

Structured pre-filter >> Hybrid Search >> LLM

Pipeline:

>> Extract Context

>> Filter Market = Vietnam

>> Retrieve Material Data

>> Hybrid Search over Ideas

>> Retrieve Market Rules

>> Rank

>> LLM

Why?

Material data has exact identifiers.

You never want embeddings deciding which SKU to retrieve.

Hybrid search combines: keyword search & semantic similarity

This improves multilingual retrieval and near-duplicate matching.

Ranking and Filtering

I would retrieve from three sources independently and merge the results.

A. Market Rules

Highest priority.

Always filtered first.

Market = Vietnam

Never retrieve Chinese rules by default.

B. Material Data

Also market-scoped.

Retrieve:

- BOM
- material thickness
- supplier PPI
- packaging specifications

These come directly from OneLake.

No semantic ranking required.

C. Prior Workshop Ideas

This is where cross-market retrieval is useful.

I would retrieve:

Top 10 Vietnam ideas and Top 5 Cross-market ideas but with lower ranking.

Ranking formula might look like:

Score = 0.50 Semantic Similarity + 0.20 Keyword Match + 0.20 Market Match + 0.10 Recency

Then apply business rules:

>> Vietnam ideas

>> Highest priority

>> Other Asian markets

>> Western Europe

Western Europe ideas should never override Vietnam-specific rules.

They are only suggestions.

Example Ranking

Suppose retrieval finds:

Idea	Market	Score
Reduce gauge 330ml	Vietnam	0.95
Thin aluminum can	Vietnam	0.90
Reduce gauge Brand C	China	0.82
Standardize cans	Germany	0.70

The model receives:

- first two
- maybe China example
- Germany only if no closer matches exist.

Retrieval Fallback

Suppose no useful results are found.

I would use a progressive fallback strategy.

Stage 1

Relax semantic threshold.

Increase Top-K.

Stage 2

Remove packaging-specific filter.

Maybe search: "Gauge reduction" instead of "330ml can gauge reduction"

Stage 3

Search cross-market ideas.

Example:

Vietnam >> Asia >> Global

Each retrieved idea is clearly marked: Cross-market reference not Applicable recommendation

Stage 4

Use structured data only.

The model still has: material master, BOM and supplier prices.

It can estimate savings, but with lower confidence.

Stage 5

Return an uncertainty message.

Example:

"No closely matching historical ideas were found for the Vietnam market. Falling back to material master data and general packaging principles. Confidence may be lower."

This is preferable to hallucinating.

End-to-End Retrieval Flow

>> User: Reduce can gauge on 330ml SKUs

>> Extract: Market=VN; Package=Can; Volume=330ml;

>> Retrieve Material Master

>> Retrieve BOM & Supplier Data

>> Retrieve Vietnam Market Rules

>> Hybrid Search: Vietnam Ideas > Optional Cross-market Ideas > Rank Results

>> LLM Generates: Savings Band, Feasibility, and One-pager

B.3 Teams Integration

The bot must handle workshop file uploads of 200–500 raw ideas. End-to-end processing takes minutes, not seconds. Design the Teams  engine interaction. Walk through:

17. How the bot acknowledges the upload and sets expectations (text reply, adaptive card, both?).
18. Where conversation and session state live, and what each persists. A short Cosmos session schema sketch is welcome.
19. How progress is shown across the minutes the user waits (streaming text, periodic updates, a single "done" ping?).
20. How the result is delivered back into the same Teams thread (adaptive card, download link, follow-up message, or similar).
21. How a user resumes a conversation across sessions (next morning, different device, same workshop).

Reference the Service Bus + ACA Worker + Cosmos pattern from the architecture above. We are looking for someone who can reason about *async-by-design*, not a synchronous chatbot. If you believe some higher-cost generation steps should happen outside the synchronous user turn, say so and explain the product implication.

ANSWER:

Overall Architecture

I would follow the asynchronous architecture already described in the case study:

- >> User uploads workshop file
- >> Teams Bot
- >> Bot validates request → Stores conversation reference → Creates session in Cosmos → Publishes job to Service Bus
- >> Immediate acknowledgement
- >> ACA Worker processes file
- >> AI Foundry + Retrieval + OneLake
- >> Results stored in Cosmos
- >> Bot posts results back to the same Teams thread

This keeps the Teams interaction responsive while allowing the backend to process large workloads.

Upload Acknowledgement

The user should receive an immediate response containing both a text message and an Adaptive Card.

Text Message: "Your workshop file has been received. Processing has started and usually takes 2–5 minutes. You'll receive a notification in this conversation when the analysis is complete."

Adaptive Card: The card should display:

Job ID

File name

Number of detected ideas

Current status

Estimated completion time

Using an Adaptive Card allows the bot to update the same card instead of creating many messages.

Conversation and Session State

I would separate conversation state from processing state.

Conversation State

Stores information required to communicate with Teams.

```
{
  "conversation_id": "...",
  "channel_id": "...",
  "tenant_id": "...",
  "user_id": "...",
  "conversation_reference": "...",
  "last_activity": "...",
  "locale": "vi"
}
```

Purpose:

- Resume conversations
- Send proactive messages
- Continue threads

Processing Session

Stored in Cosmos.

```
{
  "session_id": "VN_2026_00124",
  "market": "VN",
  "uploaded_by": "SME_VN_03",
  "status": "Processing",
  "created_at": "...",
  "updated_at": "...",
  "input_file": "...",
  "idea_count": 312,
  "progress": {
    "completed_steps": [
      "translation",
      "deduplication"
    ],
    "current_step": "feasibility",
    "percentage": 45
  },
  "result_location": "...",
  "worker_id": "...",
  "conversation_reference": "...",
  "error": null
}
```

This separates messaging concerns from business processing while providing a single source of truth for job state.

Progress Updates

I would avoid streaming LLM output.

Unlike conversational chat, workshop processing consists of discrete pipeline stages.

Instead, I would update progress when each major stage completes.

Example:

>> File uploaded
>> Translation completed
>> Duplicate detection completed
>> Savings estimation completed
>> Feasibility scoring completed
>> Generating one-pagers...

The Adaptive Card would update after each stage.

Advantages:

- Lower messaging overhead
- Clear progress
- Easier retry and monitoring
- Better alignment with asynchronous processing

Delivering Results

When processing finishes, the bot should proactively send a message into the same Teams thread.

The message would contain:

- Success notification
- Summary statistics
- Download links
- Suggested next actions

The Adaptive Card could also provide actions such as:

- Review flagged ideas
- Download Excel
- Open one-pagers
- Submit feedback

Resuming the Conversation

Since conversation state is stored in Cosmos, users should be able to resume work the next day or from another device. Because the session persists in Cosmos, the conversation continues without reprocessing the entire file.

High-Cost Processing Outside the User Turn

I would intentionally move expensive operations outside the synchronous request.

These include:

- generating multiple one-pager variants,
- cross-market benchmarking,
- creating executive summaries,
- generating alternative ideas,
- post-processing analytics.

The synchronous pipeline should focus on producing the primary deliverables needed by the user.

Secondary artifacts can be generated asynchronously after the main workflow completes.

For example:

```
>> Upload
>> Core analysis
>> Results returned
>> Background job
>> Generate executive summary
>> Generate alternative ideas
>> Refresh embeddings
>> Compute analytics
```

This improves perceived responsiveness while reducing latency for the initial interaction.

B.4 Cold-Start and Market Memory

Vietnam has near-zero historical rejection data at launch. China has more, captured ad-hoc by an SME and team via Teams conversations. Group VIPEX (Western Europe) has years of rejection history but in a different commercial culture.

Design how the agent (a) accepts new tacit rules via conversation with a market SME, (b) persists them with provenance and temporal validity, (c) applies them to feasibility scoring and rejection-aware generation, and (d) avoids leaking inappropriate WE rules into VN/CN

scoring. Sketch the market-rule schema and the application logic — pseudocode or a short example is fine. Address one additional risk explicitly: historical rejection data is useful, but it can also encode organizational inertia. How would you separate hard local constraints from contingent business resistance?

ANSWER

Design Philosophy

I would treat market rules as versioned knowledge assets, not prompt text.

The LLM's role is to extract rules from conversations, while deterministic services are responsible for validation, storage, governance, and retrieval. This separation keeps the system flexible while maintaining auditability.

Overall Workflow

- >> SME Conversation
- >> LLM detects candidate rule
- >> Extract structured fields
- >> Human review / approval
- >> Versioned rule stored
- >> Rule indexed
- >> Applied during retrieval
- >> Used by feasibility scoring and idea generation

(a) Accepting New Tacit Rules

Suppose an SME says:

"For Brand C in China, cartons cannot be replaced by shrink trays because they are also used as promotional displays."

The LLM should recognize that this is not ordinary conversation but a reusable business rule.

It extracts:

```
{  
  "market": "CN",  
  "category": "Packaging",  
  "constraint": "Do not replace Brand C cartons with shrink trays",  
  "rationale": "Carton doubles as promotional display",  
  "confidence": 0.94  
}
```

Rather than writing directly to the KB, the rule is submitted for SME review.
This prevents accidental or hallucinated rules from entering production.

(b) Rule Persistence

Each rule should contain sufficient metadata to support auditing, versioning, and expiration.

Market Rule Schema

```
{  
  "rule_id": "R_0105",  
  "market": "VN",  
  "brand": "Brand_A",  
  "category": "Packaging",  
  "constraint": "330ml cans must not be reduced below 0.235 mm",  
  "rationale": "Current supplier qualification",  
  "rule_type": "Technical",  
  "confidence": 0.95,  
  "captured_from": "Teams Conversation",  
  "captured_by": "SME_VN_04",  
  "captured_at": "2026-07-15",  
  "approved_by": "Packaging Lead",  
  "review_status": "Approved",  
  "valid_from": "2026-07-15",  
  "valid_to": "2027-07-15",  
  "version": 3,  
  "supersedes": "R_0098"  
}
```

(c) Applying Rules

When a new idea arrives:

"Reduce 330ml can gauge."

The pipeline becomes:

```
>> Extract Market  
>> Retrieve Material Data  
>> Retrieve Market Rules  
>> Retrieve Similar Rejections  
>> LLM
```

The LLM never searches the entire knowledge base itself.
Instead, deterministic retrieval provides only the relevant rules.

Example:

```
rules = get_rules(  
    market="VN",  
    category="Packaging",  
    brand="Brand_A",  
    valid_on=today  
)
```

```
history = get_rejections(  
    market="VN",  
    package="Can"  
)
```

```
prompt = build_prompt(  
    idea,  
    rules,  
    history,  
    material_data  
)
```

```
response = llm(prompt)
```

This keeps retrieval deterministic and the reasoning explainable.

Applying Rules to Feasibility

Suppose retrieval returns:

VN Rule: Minimum gauge: 0.235 mm

Idea: Reduce to 0.220 mm

The model should produce something like:

Feasibility: 38/100

Reason: Violates current Vietnam packaging rule requiring minimum gauge of 0.235 mm.

The explanation references the retrieved rule, improving transparency.

Rejection-Aware Idea Generation

The same rules should influence generation.

Instead of generating: Reduce carton >> Rejected

The LLM should generate the alternative:

Simplify carton artwork >> Lower ink usage >> Reduce adhesive

Thus, rules don't only reject ideas—they guide the model toward feasible alternatives.

(d) Preventing Western Europe Rules from Leaking

This is the most important design consideration.

I would implement strict market-scoped retrieval.

Retrieval order:

>> Vietnam Rules

>> Vietnam Historical Rejections

>> Vietnam Material Data

>> Asian Markets (optional)

>> Western Europe (reference only)

Western Europe rules should never be treated as authoritative.

Instead, they are tagged as: Reference Knowledge; Not Applicable Rule

The LLM prompt should explicitly distinguish between them.

For example:

Primary Constraints: Vietnam

Secondary References: Western Europe

Only use Western Europe examples if they do not conflict with Vietnam rules.

This prevents cultural or regulatory assumptions from leaking across markets.

Separating Hard Constraints from Organizational Inertia

Historical rejection data is valuable—but it can reinforce outdated habits.

For example:

Rejected because "We've always done it this way."

That should not become a permanent rule.

I would classify every rule into one of two categories.

Hard Constraints AND Objective constraints that are unlikely to change:

- Regulatory requirements
- Technical limitations
- Safety requirements
- Supplier compatibility
- Packaging specifications

These should strongly influence feasibility scoring.

Soft Constraints

Business preferences that may evolve:

- Brand preferences
- Marketing decisions
- Organizational habits
- Temporary budget constraints
- Current commercial strategy

These should reduce feasibility but should not block idea generation entirely.

Rule Lifecycle

To avoid stale knowledge, every rule should follow a governance process.

>> Conversation

>> Candidate Rule

>> Human Review
>> Approved
>> Active
>> Periodic Review
>> Renew / Modify / Expire

Rules with a valid_to date automatically enter a review queue before expiry, ensuring that temporary commercial decisions do not silently become permanent knowledge.

B.5 Multilingual Routing & Cost Envelope

VN and CN users converse in local language. Material masters are in English. Outputs return in local language for users; one-pagers for EVP forums are in English.

Show how the agent routes language across (a) query understanding, (b) retrieval — mixed-language index? translate-then-query? query-then-translate? — (c) generation, and (d) final rendering.

Sketch a latency and cost envelope for a typical 300-idea workshop file. Reference benchmark: per-market opex budget is ~50,000 DKK / 10 active users / year on a current GPT-class deployment. End-to-end target: under two minutes per workshop file.

ANSWER

Design Principles

My design is based on four principles:

- Keep enterprise knowledge in one canonical language (English).
 - Retrieve using canonical English rather than maintaining separate indexes per language.
 - Translate only where necessary.
 - Process workshop files in parallel rather than sequentially.
-

This minimizes maintenance effort, improves retrieval consistency, and reduces operational cost.

Overall Language Flow

- >> Vietnamese / Chinese User
- >> Language Detection
- >> Translate to Canonical English
- >> Structured Extraction
- >> Retrieve English Knowledge Base
- >> LLM Reasoning (English)
- >> Generate Outputs: English One-pager
- >> Translate User Response
- >> Vietnamese / Chinese

English becomes the system's internal reasoning language, while users continue interacting in their preferred language.

(a) Query Understanding

The first step is language detection.

Example for Vietnamese input:

The pipeline performs:

- >> Detect Language
- >> Vietnamese
- >> Translate to English: "Reduce 330ml can gauge"
- >> Extract Entities

Structured entities:

```
{  
  "market": "VN",  
  "package": "330ml can",  
  "idea_type": "Gauge Reduction"  
}
```

The original text is preserved for traceability.

(b) Retrieval Strategy

Translate → Structured Filtering → Hybrid Retrieval

Pipeline:

- >> English Query
- >> Market Filter
- >> Material Master Lookup
- >> Hybrid Search
- >> Market Rules
- >> LLM

This is because Material masters already exist in English.

Maintaining multilingual copies of: BOM, Supplier PPI, Material specifications would create unnecessary synchronization problems.

Why not a multilingual vector index?

A multilingual index sounds attractive, but it introduces challenges: duplicated embeddings, inconsistent ranking, higher storage costs, more complex maintenance.

Since enterprise data is already maintained in English, translating queries into a canonical language simplifies governance while still allowing multilingual user interactions.

RETRIEVAL SOURCES

Structured Data

Retrieved directly using deterministic filters.

Examples:

SKU, Material master, BOM, Supplier PPI

No embeddings required.

Prior Ideas

Hybrid Search: keyword, semantic, market filter

Market Rules

Retrieved using: "Market = Vietnam AND Category = Packaging" before semantic ranking.
This prevents Chinese rules from influencing Vietnam recommendations.

(c) Generation

I would perform reasoning in English.

Reasons: Material data is English, Rules are stored in English, Prompt templates are simpler, Evaluation is easier.

(d) Final Rendering

Different audiences receive different languages.

Vietnam SME: Vietnamese

China SME: Chinese

EVP Forum: English

Engineering Logs: English

The system therefore performs a final translation only for user-facing responses.
This avoids translating the knowledge base itself.

Latency Envelope

Target: Under two minutes for 300 ideas

I would process the workflow as a pipeline rather than sequentially.

Parallel Processing

A 300-idea workshop should never be processed by one massive LLM request.

Instead:

>> 300 Ideas

>> 15 batches -- 20 ideas each

>> Parallel ACA Workers

>> Merge Results

Advantages: better throughput, lower latency, improved retry behavior, avoids token limits.

Cost Envelope

The target budget is:

50,000 DKK/year for 10 active users

≈ 5,000 DKK per user per year

To remain within this budget, I would optimize costs through:

1. Minimize LLM Calls

Translate and normalize once per idea.
Avoid repeated translations during retrieval.

2. Deterministic Retrieval

Material lookup, BOM lookup, supplier lookup, and filtering should use deterministic services instead of LLM calls.

3. Batch Processing

Translate and score ideas in batches rather than invoking the model individually for every idea.

4. Cache Results

Reuse: translated ideas, embeddings, duplicate clusters, retrieved market rules, across retries and follow-up conversations.

5. Tiered Model Usage

Use the GPT-class model only where reasoning is required.
Do not use it for: filtering, lookups, ranking, session management.

This reduces token consumption significantly.

Additional Optimization

Some expensive operations do not need to block the user's workflow.

I would defer: generation of alternative ideas, executive summaries, cross-market benchmarking, analytics dashboards to background jobs after the primary analysis completes.
This improves responsiveness while keeping operating costs predictable.

Part C: Production Boundaries & Release Gates

C.1 Sample End-to-End Workflow

Illustrate one plausible end-to-end workflow for VIPEX Coach using either pseudocode, a diagram, or both. Pick one representative path such as:

- workshop-file upload -> cleaning / deduplication -> savings sizing -> feasibility scoring -> one-pager generation
- idea generation -> retrieval / grounding -> rule application -> savings / feasibility checks -> surfaced artifact

Your illustration should show:

22. the major stages in order;
23. which stages you would implement primarily as deterministic logic vs model-driven reasoning;
24. where key data or memory enters the flow (for example material data, prior ideas, market rules, session state);
25. one or two failure points you would expect in that workflow;
26. what telemetry or eval would help you detect those failures quickly.

We are not grading visual polish. We are grading whether your workflow makes the system legible and operationally debuggable.

ANSWER

End-to-End Workflow

```
>> User uploads workshop file
>> Teams Bot validates upload (Deterministic Logic)
>> Create Session (Cosmos DB)
>> Publish Job (Service Bus)
>> ACA Worker picks up the job
>>
```

Stage 1: Translation & Normalization (Model + Deterministic Validation)

Input:

- Raw workshop ideas
- Session state

Output:

- Clean English ideas
 - Structured metadata
-

Stage 2: Deduplication (Model + Embedding Search)

Uses:

- Prior workshop ideas
- Vector index

Output:

- Unique idea list
-

Stage 3: Structured Retrieval (Deterministic)

Retrieve:

- Material Master
 - BOM
 - Supplier PPI
 - Market Rules
 - Prior accepted ideas
-

Stage 4: Savings Estimation (Deterministic Service)

Uses:

- Material cost
- BOM
- Supplier pricing

Output:

- Low / Mid / High savings
-

Stage 5: Feasibility Scoring (Model Reasoning)

Uses:

- Retrieved market rules
- Historical rejections
- Savings estimate

Output:

- Score
 - Confidence
 - Explanation
-

Stage 6: One-Pager Generation (Model)

Uses:

- Clean idea
- Savings
- Feasibility
- Supporting evidence

Output:

- Executive summary
 - Implementation brief
-

>> Persist Results (Cosmos)

>> Teams Bot sends Adaptive Card

>> User Review & Feedback

Deterministic vs Model Responsibilities

Stage	Primary Implementation
Upload validation	Deterministic
Session creation	Deterministic
Translation & normalization	Model
Entity extraction	Model
Material lookup	Deterministic

BOM lookup	Deterministic
Market rule retrieval	Deterministic
Duplicate detection	Model + Embeddings
Savings calculation	Deterministic
Feasibility scoring	Model
One-pager drafting	Model
Persist session	Deterministic
Teams notification	Deterministic

This separation keeps financial calculations and enterprise data access deterministic while allowing the LLM to focus on language understanding and reasoning.

Data and Memory Flow

Different knowledge sources enter at different stages.

Session State (Cosmos)

Used throughout the workflow for: user identity, conversation continuity, progress tracking, recovery after failures.

Material Master / BOM / Supplier PPI

Injected before savings estimation.

These provide the structured financial inputs used by deterministic calculations.

Market Rules

Retrieved before feasibility scoring.

Only rules from the active market are considered authoritative.

Prior Workshop Ideas

Retrieved during deduplication and idea grounding.

Cross-market ideas are treated as references rather than constraints.

Historical Rejections

Used during feasibility scoring to explain likely rejection risks and improve recommendation quality.

Expected Failure Points

Failure Point 1 — Poor Retrieval

Suppose retrieval misses an important Vietnam market rule.

Example:

- >> Reduce carton
- >> Relevant Vietnam rule not retrieved
- >> LLM recommends invalid idea

Telemetry

Monitor: retrieval precision, retrieval recall, percentage of prompts with zero relevant market rules, average retrieval confidence.

These metrics help detect indexing or filtering regressions.

Failure Point 2 — Feasibility Score Drift

Suppose the LLM begins assigning overly optimistic feasibility scores.

Example:

- >> Predicted 92
- >> Business rejects 80% of ideas

The retrieval layer is correct. The reasoning layer has drifted.

Telemetry

Track: feasibility score distribution, acceptance rate by score bucket, calibration error, score variance for repeated evaluations.

A significant drop in calibration indicates that prompts or model behavior should be reviewed.

Operational Telemetry

I would instrument each stage independently rather than relying only on end-to-end metrics.

Stage	Metrics
Upload	Upload success rate, file validation failures
Translation	Translation confidence, unsupported-language rate
Deduplication	Duplicate precision/recall, duplicate ratio
Retrieval	Retrieval latency, recall, zero-hit rate
Savings	Missing material data %, calculation failures
Feasibility	Score distribution, acceptance calibration
One-pager	Generation latency, regeneration rate
Delivery	Teams notification success, total workflow latency

Stage-level telemetry allows regressions to be localized quickly.

Evaluation Strategy

In addition to operational metrics, I would maintain a golden evaluation set for each stage:

Translation: compare against manually verified translations.

Deduplication: precision and recall against a labeled duplicate dataset.

Retrieval: recall@K for market rules and historical ideas.

Savings estimation: compare against finance-approved estimates.

Feasibility scoring: calibration against historical accept/reject outcomes.

One-pager generation: SME quality ratings and edit distance from final approved documents.

This allows individual stages to be evaluated independently before changes reach production.

C.2 Hardening Priorities

You only have capacity this quarter to harden **two** areas of the system into more controlled services or workflows. Which two would you choose, and why? What is the strongest counter-argument to your prioritization?

ANSWER

I would harden two components:

(1) the Knowledge Retrieval & Market Rule Service and

(2) the Savings Estimation Service.

The Knowledge Retrieval service is the foundation of the entire AI workflow. It should provide deterministic, market-scoped access to material masters, BOMs, supplier PPI, historical ideas, and versioned market rules. By enforcing structured filtering before semantic retrieval and continuously evaluating retrieval quality, this service ensures that every downstream LLM decision is grounded in the correct context.

The Savings Estimation service should also be deterministic because it produces financial outputs that directly influence business decisions. Moving calculations into a controlled service makes the results reproducible, auditable, and explainable, while allowing the LLM to focus on interpreting and communicating those results rather than computing them.

The strongest counter-argument is that operational orchestration should be hardened first. If Service Bus processing, ACA Workers, or Teams callbacks are unreliable, users may never receive results regardless of model quality. This is a valid concern, especially during an early pilot. However, I believe cloud-native orchestration failures are generally easier to observe and recover from than subtle errors in retrieval or financial reasoning. Strengthening retrieval and savings estimation establishes trusted decision-making foundations while allowing orchestration reliability to improve through operational monitoring and incremental engineering.

C.3 Release Gates

Before locking the chosen architecture in for the China launch and GCC handover, name three to five **measurable gates** the Vietnam pilot must pass. Each gate must be (a) computable from production telemetry or eval results, (b) bounded with a numeric threshold, (c) defensible at a steerco meeting.

ANSWER

Gate	Metric	Threshold	Why it Matters
Business Acceptance	Commercial acceptance rate	≥70%	Demonstrates that the AI's recommendations are trusted and actionable.
End-to-End Performance	P95 processing latency for a 300-idea workshop	<2 minutes	Confirms the system meets the product's SLA and provides a responsive user experience.
Retrieval Quality	Market-rule Recall@5	≥95%	Ensures the agent consistently retrieves the correct market-specific knowledge before reasoning.
Savings Estimation Accuracy	Median deviation between predicted mid-point savings and finance-approved estimate	≤10%	Validates that financial recommendations are sufficiently accurate for business decision-making.
Operational Reliability	Successful workflow completion rate	≥99%	Demonstrates that uploads complete successfully without manual intervention or recovery.

Part D: Evaluation Framework

D.1 Idea Quality vs Engine Quality

Distinguish between **idea quality** (is the agent producing *good* cost-saving ideas?) and **engine quality** (is the agent performing as a *system*?). Propose at least three metrics for each, with a one-line note on how each would be computed.

Anchors to extend or replace:

- *Idea quality*: savings-band realism (predicted band vs realised savings six months later), feasibility-score calibration (predicted score vs actual reject / accept rate), translation fidelity (human spot-check on meaning preservation and packaging-term accuracy over a sampled set), dedupe precision and recall against a held-out set.
- *Engine quality*: P50 / P95 latency end-to-end, P99 cost per workshop file, tool-call success rate, conversation completion rate, market-rule application rate.

ANSWER:

Idea Quality Metrics

Metric	How Computed	Why It Matters
Savings Band Accuracy	Compare predicted low/mid/high savings with realized or finance-approved savings after implementation.	Measures whether the AI's financial estimates are realistic.
Feasibility Score Calibration	Compare predicted feasibility score with actual acceptance/rejection rate from commercial steerco.	Ensures a score of 80 actually corresponds to a high likelihood of approval.
Translation Fidelity	Human evaluation of sampled translations for meaning preservation and packaging terminology accuracy.	Prevents incorrect recommendations caused by translation errors.
Duplicate Detection Precision & Recall	Evaluate against a manually labeled duplicate dataset.	Ensures duplicate ideas are merged without incorrectly collapsing distinct ideas.
Business Acceptance Rate	Percentage of AI-generated proposals accepted by the commercial steering committee.	Direct measure of business usefulness and trust.

Engine Quality Metrics

Metric	How Computed	Why It Matters
P95 End-to-End Latency	Time from Teams upload to final response.	Measures responsiveness under realistic workloads.
Workflow Completion Rate	Successful completed jobs ÷ total submitted jobs.	Indicates operational reliability.
Tool Call Success Rate	Successful retrieval/tool invocations ÷ total tool calls.	Detects failures in retrieval or enterprise integrations.
P99 Cost per Workshop File	Compute cost of processing each file; monitor the 99th percentile.	Ensures operating costs remain within budget.
Market Rule Application Rate	Percentage of applicable market rules successfully retrieved and used during reasoning.	Measures correct grounding before generation.

D.2 Per-Step Eval for the Chosen Architecture

Use the boundary choices you made in Part C. Where would you instrument eval scoring — per stage, per tool call, or end-to-end only? Show that your design is **eval-tractable**: if a regression in feasibility scoring lands tomorrow, can you localise it within hours, or do you have to inspect broad end-to-end traces? Be specific about which telemetry, traces, or eval slices you would inspect first.

This question is deliberately a stress test on the architecture you picked. If it's not tractable, say so — that's a useful answer.

ANSWER:

The architecture deliberately separates deterministic services (retrieval, material lookup, savings estimation) from model-driven reasoning (translation, feasibility scoring, one-pager generation). Because every stage emits structured telemetry and versioned traces, regressions can be localized quickly.

For example, if feasibility scoring degrades tomorrow, I would not inspect broad end-to-end logs first. Instead, I would:

1. Check **feasibility calibration** to confirm the regression.
2. Verify **market-rule retrieval quality** (Recall@5, zero-hit rate, retrieved rule count).

3. Confirm the **savings service** and structured data outputs are unchanged.
4. Compare **prompt and model versions** against the previous deployment.
5. Review sampled reasoning traces only if earlier stages appear healthy.

This layered instrumentation allows most regressions to be isolated within hours rather than requiring manual inspection of complete workflow traces. End-to-end metrics remain important for measuring business outcomes, but they are insufficient for diagnosing failures. By instrumenting every stage and every critical tool call, the system remains observable, debuggable, and safe to evolve in production—exactly the properties expected for an enterprise AI platform.

D.3 Golden Idea Set

Design a ten-row golden set for the Vietnam pilot. Each row should have: raw idea text (in Vietnamese), expected cleaned text (in English), expected savings band, expected feasibility score, expected market-rule hits (if any), source workshop reference. One row is filled below as a worked example; add the rest, or describe how you would generate them.

#	Raw (VN)	Cleaned (EN)	Savings band (DKK / yr)	Feasibility (0-100)	Market-rule hits	Workshop
1	Giảm độ dày lon 330ml xuống 0.235mm	Reduce 330ml can gauge to 0.235mm	1.8M – 2.6M	72	(none)	VN Walk-the-Floor 2026-W22
2						
3						
4						
5						
6						
7						
8						
9						
10						

ANSWER:

The initial 10 rows would only be the seed dataset.

In production, I would grow the golden set continuously from real workshops:

- >> Production Workshops
- >> SMEs identify interesting ideas
- >> Finance validates savings
- >> Commercial team records accept/reject
- >> Packaging experts validate translations
- >> Golden Set Version 2

Every row would therefore contain ground truth from actual business decisions rather than synthetic estimates.

This way I would expand my golden dataset into a continuously versioned evaluation dataset built from real Vietnam workshops, where every row is labeled by SMEs and Finance. The Golden Set would become part of the CI/CD pipeline, so every model, prompt, retrieval, or rule change is automatically evaluated before release. This makes regressions immediately visible and provides a reproducible benchmark for future market rollouts such as China.

D.4 Drift Detection

How would you detect that the engine is becoming **less helpful over time**? Consider: operator acceptance rate trending down, savings-band P50 misses widening, market-rule application precision degrading. Reference rule *temporal validity* — rules can expire silently. Propose three drift signals and the alert thresholds you would attach.

ANSWER:

I would monitor three complementary drift signals: business acceptance, prediction accuracy, and knowledge freshness. Business acceptance is measured using the rolling 30-day commercial acceptance rate, with an alert triggered if it drops by more than 15 percentage points from the historical baseline or falls below 60%. This is the strongest indicator that the system is becoming less useful to its users.

Second, I would monitor savings estimation drift by comparing predicted midpoint savings with finance-approved or realized savings using Median Absolute Percentage Error (MdAPE). An alert would trigger if the rolling MdAPE exceeds 15% or increases by more than 5 percentage points compared with the previous quarter, indicating that cost models or enterprise data may have become outdated.

Finally, I would monitor market-rule drift, recognizing that market rules have temporal validity. I would track both market-rule application precision and the percentage of applied rules that have expired. An alert would trigger if rule application precision falls below 95% or if more than 5% of applied rules are past their validity period. This ensures that silently expired rules do not continue influencing feasibility scoring or idea generation.

Together, these three signals cover the business, predictive, and knowledge dimensions of drift. Because each is tied to specific telemetry and evaluation datasets, they enable targeted investigations rather than requiring broad end-to-end debugging whenever the system's usefulness declines.

D.5 In-Teams Feedback Loop

Design a low-friction feedback mechanism inside Teams (≤ 2 seconds per idea). A short adaptive-card sketch is acceptable. Show how the signal flows back into:

- 27. feasibility scoring — improves over time.
- 28. rejection-aware generation — avoid repeating rejected ideas.
- 29. The golden-set maintenance loop — new golden rows when an SME corrects an answer.

ANSWER:

I would optimize for high-volume, low-friction feedback.

A simple binary or categorical signal from thousands of interactions is more valuable than a few detailed reviews.

The feedback loop should feed three independent learning systems:

- Feasibility Scoring
- Rejection-Aware Idea Generation
- Golden Set Maintenance

Adaptive Card

After every processed idea, the user sees a compact card.

Idea #24

Reduce 330ml can gauge to 0.235 mm

Savings:

1.8M – 2.6M DKK

Feasibility:

72/100

👍 Useful

👎 Not Useful

Wrong Savings

Not Feasible

Correct Translation

[Optional Comment]

The common actions ("👍", "👎", "Wrong Savings", "Not Feasible") are one-click operations that take less than two seconds. The optional comment box is only used when users want to explain their choice.

Feedback Pipeline

>> Teams Adaptive Card

>> Feedback API

>> Cosmos Feedback Store: Feasibility Learning, Generation Learning, Golden Set Maintenance

The important point is that one click fans out into multiple learning pipelines.

Improving Feasibility Scoring

Every feedback event becomes a labeled training example.

Example:

Idea: Reduce can gauge

AI Score: 82

SME: Not Feasible

Stored as:

```
{  
  "idea_id": "VN102",  
  "predicted_score": 82,  
  "actual_label": "Rejected",  
  "reason": "Technical constraint"  
}
```

These labels are not used to fine-tune the LLM directly. Instead, they are used to:

- recalibrate score thresholds,
- evaluate calibration,
- improve prompts,
- train a future ranking model if sufficient data is collected.

Over time, the feasibility score becomes better aligned with actual commercial decisions.

Improving Rejection-Aware Generation

Suppose multiple SMEs reject similar ideas.

Example:

Idea: Replace carton with shrink wrap

Rejected Reason: Brand display requirement

The rejection becomes structured knowledge.

```
{  
  "market": "VN",  
  "idea_type": "Packaging",  
  "reason": "Brand Display",  
  "status": "Rejected"  
}
```

Future generation retrieves this history.

Instead of proposing: Replace carton

The model generates:

Reduce ink usage

or

Simplify carton design

The agent learns what not to suggest without permanently banning the idea.

Golden Set Maintenance

Suppose an SME corrects:

AI: Reduce can gauge

SME: Actually this means reduce lid weight.

That correction is valuable.

Rather than simply fixing today's answer,

I would send it to an evaluation queue.

>> SME Correction

>> Candidate Golden Example

>> Human Review

>> Golden Set

Only high-quality corrections become new golden rows.

This keeps the evaluation dataset growing with real production examples.

Feedback Schema

Every feedback event should capture more than a thumbs-up/down.

Example:

```
{  
  "idea_id": "VN102",  
  "session_id": "VN_W22",  
  "feedback_type": "Not Feasible",  
  "corrected_text": null,  
  "user_comment": "Supplier cannot manufacture below 0.235 mm",  
  "market": "VN",
```

```
"user_role": "Packaging SME",  
"timestamp": "...",  
"model_version": "GPT5.2",  
"prompt_version": "P17"  
}
```

Including the model and prompt versions makes it possible to analyze regressions after deployments.

Governance

Not every piece of feedback should immediately change system behavior.

I would separate:

Operational Feedback: Used immediately for dashboards.

Example:

- Helpful
- Not Helpful
- Wrong Translation

Knowledge Feedback: Requires SME review before updating market rules.

Evaluation Feedback:

- Candidate additions to the Golden Set.
- Requires QA approval.
- This prevents noisy user clicks from corrupting the knowledge base.

Reference the Teams adaptive-card pattern from B.3.

Synthetic Data Appendix

Use this data to support your answers in Parts B and D. All values are synthetic — no real customer, SAP, or supplier identifiers.

Sample Raw Workshop Ideas

```
idea_id,market,workshop,language,raw_text,submitted_by
idea_001,VN,Walk-the-Floor 2026-W22,vi,"Giảm độ dày lon 330ml xuống 0.235mm",SME_VN_03
idea_002,VN,Walk-the-Floor 2026-W22,vi,"giam can gauge 330ml",SME_VN_07
idea_003,VN,Walk-the-Floor 2026-W22,vi,"Bỏ in nhãn màu vàng trên thùng carton 24 lon",SME_VN_03
idea_004,VN,Walk-the-Floor 2026-W22,en,"Switch shrink film from PE to bio-PE on 500ml HDPE",SME_VN_11
idea_005,CN,Gemba Kaizen 2026-W23,zh,"330 毫升铝罐颈部规格统一为 202 直径",SME_CN_02
idea_006,CN,Gemba Kaizen 2026-W23,zh,"330ml 罐 颈口标准化 202",SME_CN_05
idea_007,CN,Gemba Kaizen 2026-W23,zh,"取消彩盒, 改为简装托盘",SME_CN_02
idea_008,CN,Gemba Kaizen 2026-W23,zh,"啤酒瓶颈玻璃克重从 145g 降到 138g",SME_CN_08
idea_009,VN,Walk-the-Floor 2026-W22,vi,"Đổi nắp khoén 26mm sang 24mm",SME_VN_11
idea_010,CN,Gemba Kaizen 2026-W23,zh,"夏季出货取消热缩膜",SME_CN_05
```

Notes:

- idea_001 and idea_002 are near-duplicates ("reduce 330ml can gauge"). idea_005 and idea_006 are near-duplicates ("standardise neck diameter to 202").
- idea_007 ("remove colour carton, use simple tray") is a strong candidate for a market-rule hit — see synthetic rule R_002 below.
- idea_004 is submitted in English even though the workshop was in Vietnam.

Sample Material Master Extract

```
sku_code,market,brand,packaging_component,gauge_mm,weight_g,unit_cost_dkk,supplier_anon,currency
SKU_VN_3301,VN,Brand_A,can_330ml,0.250,12.4,0.412,Supplier_A1,DKK
SKU_VN_3302,VN,Brand_A,can_330ml,0.250,12.4,0.412,Supplier_A1,DKK
SKU_CN_3301,CN,Brand_A,can_330ml,0.235,11.8,0.398,Supplier_C1,DKK
SKU_CN_5001,CN,Brand_B,bottle_500ml,—,145.0,0.987,Supplier_C2,DKK
SKU_VN_5001,VN,Brand_B,bottle_500ml,—,138.0,0.961,Supplier_A2,DKK
SKU_CN_3302,CN,Brand_C,can_330ml,0.235,11.8,0.398,Supplier_C1,DKK
```

Sample Rejection History

```
idea_ref,market,rejection_category,rejection_reason,rejected_at
idea_h_001,CN,brand,"Carton serves dual purpose as in-store display; tray substitution rejected by Brand_C marketing",2025-09-12
idea_h_002,VN,technical,"Cap diameter change requires new filling-line tooling; not in 2026 capex plan",2025-11-04
idea_h_003,CN,commercial,"Cannot reduce gauge below 0.235mm on Brand_C SKUs (consumer perception of premium)",2024-06-18
idea_h_004,MY,brand,"Colour change on glass bottle rejected – Muslim market constraints (Brand_B premium variant)",2025-02-22
```

Sample Market Memory Rules

```
[
  {
    "rule_id": "R_001",
    "market": "CN",
    "category": "labeling",
    "constraint": "Label wort concentration in degrees Plato, not ABV percentage",
    "rationale": "Chinese regulation labels alcoholic strength by wort, not finished-product ABV. Commercial team rejects ideas that compare to ABV-labelled markets without conversion.",
    "captured_by": "SME_CN_02",
    "captured_at": "2026-03-12",
    "valid_from": "2026-03-12",
    "valid_to": null,
    "source_conversation_id": "thread_CN_0334"
  },
  {
    "rule_id": "R_002",
    "market": "CN",
    "category": "packaging",
    "constraint": "330ml carton on Brand_C SKUs cannot be substituted with shrink-tray packaging",
    "rationale": "Carton serves dual purpose: secondary packaging plus in-store promotional display. Brand_C marketing rejects tray substitutions categorically.",
    "captured_by": "SME_CN_05",
    "captured_at": "2026-02-04",
    "valid_from": "2026-02-04",
    "valid_to": "2026-12-31",
    "source_conversation_id": "thread_CN_0298"
  }
]
```

R_002 has a `valid_to` of end-of-year — the agent must resurface it in Q4 2026 for review (Brand_C may revisit).

Submission Instructions

- Submit a single document (Markdown, Word, or PDF).
- Include diagrams or pseudocode inline where they support your answer.
- Note actual time spent per section.
- Total target: ~2.25 hours. If you ran over significantly, note where and why — that's a useful signal.

Questions before you start: contact the hiring manager.

Good luck.